Exercise 1

Introducing Nodes and Queues
(and Menus and Modes)

In this exercise, you will create two nodes, a generic `controller` and an `actuator` and make them communicate through a command queue at the press of a button.

(If you missed the presentation on the basics of the GöROS framework, you can find the slides on StudIP and read through the introduction section of the documentation.)

Set-up

You can run the framework locally or on the raspberry pi by downloading the code from the course gitlab. You can use the `requirements.txt` in the gitlab repository to set up a Python `venv` and/or to make sure your pi has all necessary Python packages installed.

Unfortunately, you might not be able to run the framework locally on Windows, at least not without a Linux emulator. It *might* work with a venv, but we ran into problems with the key listener (specifically the `termios` module) during testing on Windows. Likewise, `conda` (as an alternative to `venv`) also caused some issues that we haven't been able to track down yet, so we recommend you use a python `venv` on Linux if you want to run the code locally.

At the end of Exercise 2, we will give you instructions for how to modify the config files in the framework to run a minimal implementation on the raspberry pi that allows you to connect the camera to the viewer and to manually drive the robot using `WASD`-keys.

Task 1.1: Creating Nodes

Every node has its own python file which needs to be saved inside the `nodes` folder. To create your own nodes, you can copy `template_node.py` and simply change the class name.

The name for each node has to be unique, and the convention is to have a general name (`"main"`, `"camera, "ev3"`) that is used to identify the node in configs or when sending commands to a node. By convention, these names are lowercase and the file is given a name like `main_node.py`, `camera_node.py` etc.

Inside the file, the node's class name is defined, conventionally using CamelCase:

```
class CameraNode(Node):
```

None of these naming conventions are integral to the functioning of the robot, just make sure you know which class- and file- name go with which node name and that all of them are unique.

While every node has code that is unique to it, they also share a lot of methods that handle things like communication among nodes, publishing to and reading messages from topics, or an orderly shut-down. These shared functions are inherited from the basic `Node` class, and you can find a list and brief description of all of them in the technical reference at the end of the documentation.

(If you are rusty on the basics of OOP and inheritance in Python, now would be a good time to brush up!)

Your Task:

- Create two new nodes (e.g. `sender` & `listener` or `controller` & `actuator`) by copying `template_node.py` inside the `nodes` folder and updating the file and class names accordingly.

Task 1.2: The Node Config

The information about which nodes your robot should use when you start the framework and how they are connected with each other is contained in the file `node_config.yaml`. (All configs can be found in the `configs` folder.)

As an example, the config for a `vision` Node might look something like this:

```yaml
  vision:
    class_name: VisionNode
    queues: ["camera"]
    topics:
      pubs: ["marker_detections"]
      subs: ["camera_capture", "robot_pose"]
```

The first part of the config are the node name (`"vision"`) and class name (`"VisionNode"`) we discussed in Task 1.1. If you don't give a class name here or there is no class of the name you specified anywhere in the `nodes` folder, you will get a warning. The framework will still boot up, but it will use the parent `Node` as the class for this node. That means it has all the basic functions like `ping()` and `switch_mode()` and `stop()`, but if you send it any commands beyond the basics, they won't have any effect beyond being printed to the console.

The next bit of information in the config is the list of `queues`. Nodes can send commands to other nodes by using queues (more on this in Task 1.3). In order to have access to another node's queue, the recipient's name needs to be added to the sender's `queues` list. In the example above, the `vision` node can send commands to the `camera` node. If your node doesn't need to send commands, leave the list empty.

Finally, the config has lists of topics a node publishes and subscribes to, but these won't become relevant until Exercise 2.

In addition to the information on individual nodes, the node config also contains a list called `include`. Any node you want to be initiated when you start the framework needs to be listed here.

```yaml
include:
  # SYSTEM NODES (always required)
  - main # main node always needs to be included!
  # CUSTOM NODES
  - robot_controller
  - ev3
  - camera
  # Add your own nodes below
```

Running the framework like this means that in addition to the mandatory `main` node, you also get the `robot_controller` node, the `camera` node, and the `ev3` node.

Not every node that's defined in the config needs to be included, so you can easily swap nodes in and out depending on your use case, without having to first delete or re-enter all of their

information. On the flip side, if a node listed under `include` does not have its own config, it will be instantiated as a dummy node (basic `Node` class), the same way as happens when the class name is missing.

Your Task:

- Add the two nodes you created to the `node_config`. You can use the template at the top of the file.
- You want the `controller` to send commands to the `actuator`, so add that information to the config.
- Add your nodes to the list of nodes to be included in the framework.

At this point, you can fire up the framework by navigating to the `robot_code` directory and running `main.py`. If you've done everything correctly, you should get notifications that your nodes are up and running, but so far, nothing else is going to happen. So, you can shut the program down again by pressing the `x` key on your keyboard.

Task 1.3: Simple Commands

Each node can perform its functions in different ways. The `run_step()` method that is included in the `template` node is part of the run loop and will be explored in Exercise 2. For now, we are going ignore it and focus on writing commands that we can execute directly.

For this task, you will write one function for each of your nodes.

Write a function for the `actuator` node that takes a message (e.g. a string) as a parameter and prints that message to the console, something along the lines of:

```
def print_message(self, msg):
  print(msg)
```

We want the `controller` to trigger the `actuator`'s printing-method. Since all nodes are instances of Python's `multiprocessing.Process`, they are somewhat insulated from each other and can't just call functions in another node directly. That means there is no way to write a function for the controller that looks like this:

```
def send_print_message(self, recipient, msg):
  recipient.print_msg(msg)
```

Instead, nodes use the queues mentioned above to send commands to each other. The exact mechanism for how a node checks its command queue is handled by the base `Node` class and doesn't have to concern us here (you can read up on it in the `Node` section in the documentation). For now, we just need to know that queues can send commands one-way from sender-note to recipient-node(s), and that every node checks whether it has received a new command about ten times a second.

To call the `actuator`'s print function, the `controller` node needs to send the name of the function and any parameters the function needs (in this case: the message to be printed) through the `actuator`'s command queue. The method you need for this is already implemented in the base `Node` class and you can call it e.g. like this:

```
self.send_to_queues(
  recipients="actuator",
  msg=("print_message", "Hello, World!") # ("function name", args)
)
```

`recipients` takes either the name of a single node, a list of node names, or `"all"`.
`msg` has to be a list or tuple, containing the name of the function that the recipient(s) should execute as a string and all the arguments that function requires (or `None`). If the function needs more than one argument, they, too, get wrapped in a list or tuple. (You can find more detailed information about the `send_to_queues` function in the documentation.)

Your Task:

- Write a function for the `actuator` that receives a string as a function parameter and prints that string to the console.
- Write a function for the `controller` (e.g. `send_string`) that in turn calls `send_to_queues` to send the `print_message` command and a string to be printed to the `actuator`.

(If you are familiar with the Python module `rich`, you can use its syntax to format your message. Calling `print()` automatically calls `rich.print()` in the background.)

Important Note: We mentioned above that nodes check their queues regularly for new commands and then execute any commands they find there. The way this is implemented leads to one peculiarity: All functions triggered by a command (either through a queue or through a key press, cf. below) are self-contained, meaning you don't have access to their return values!

Say you have a node `calculator` with the method

```
def addition(self, x, y):
  return x + y
```

which you call from a controller node like this:

```
def calculate_sum(self):
  self.send_to_queues(
    recipients="calculator",
    msg=("addition", (1,2))
}
```

In this case, the `calculator` node has no way of accessing the returned `x+y` value!

If you want that result to be available to your `calculator` node outside of the `addition()` method, you need to either publish it to a topic (more on that in Exercise 2) or store it as an instance variable:

```
def addition(self, x, y):
  self.sum = x + y
```

Task 1.4: The Menu

Now we need a way to call the function you just wrote for the `controller`. The GöROS framework allows you to use keyboard input to trigger functions in specific nodes. The connections between keys and their effects are defined in a set of dictionaries inside the `__init__()` method of the `Menu` class, found in `utils/menu.py`.

For each function you want to bind to a specific key, you need to add an entry to one of these key-press dictionaries. The format is always the same: `{key: ("target_node", "command_name", arguments)}`. The formats for the target node(s) and the command name and arguments follow the same rules as `recipients` and `msg` in queues (cf. above).

For example:

```
{
  "p": (["actuator", "controller"], "ping", None), # ping-function (from base Node)
  "d": ("motor", "drive", (3,6)) # for a hypothetical function: drive(speed, turn)
}
```

When you look at `menu.py`, you will see that the dictionaries are organized around different modes (currently: `self.idle`, `self.manual`). For now, the only thing you need to know about modes is that the robot starts up in mode `IDLE`, so you need to add your key-press entries to the corresponding dictionary.

Additionally, there is also a dictionary of reserved keys: `self.reserved`. This, too, will be explained in more detail below; for now, just make sure you don't use any of the keys listed there for your own entries. (You will get an error message if you do.)

Your Task:

- Add an element to the `self.idle` dictionary that calls the function you wrote for your `controller` when a specific key is pressed.
- Run the robot framework by calling `main.py` and press your selected key. (If you accidentally hit a key that takes you to a different mode, you can return to mode `IDLE` by pressing `q`.)

(If pressing the key has no effect, check the messages printed to the console as the framework starts up. That might give you a first indication of what might have gone wrong. If you are still having trouble, you can check the section about Tips and Troubleshooting in the documentation.)

Task 1.5: Modes

Finally, we want to have a look at the concept of modes. At any given time, the framework is in one of a set number of pre-defined modes. This can affect how nodes behave, what functions are available to use, and what effect these functions have.

For example, in the standard framework, the `MANUAL` mode allows you to drive the robot using `WASD` keys. If our robot had a touch sensor, we could implement a `TOUCH` mode where the the `WASD` keys have no affect but the robot just drives straight ahead until the touch sensor is pressed and then it stops.

Modes are defined as an enum in `menu.py`. At the moment, only two modes are implemented, `Mode.IDLE` and `Mode.MANUAL`:

```
class Mode(Enum):
    IDLE = 0
    MANUAL = 1
```

but you can add as many other modes as you would like.

The logic for switching modes is implemented in the `main` node and in the `Node` base class as `Node.switch_mode()`, so the only thing you need to do is choose keys that will initiate the switch. These will then be stored in the dictionary of reserved keys in the menu: `self.reserved`. This dictionary contains functions that need to be accessible no matter what mode the robot is in (first and foremost mode-switching and shut-down).

```
self.reserved = {
    self.stop_key: ("main", "stop_all", None), # Default: 'x', can be changed in
main config
    "q": ("main", "switch_mode", (0,)), # switch to idle mode
    "m": ("main", "switch_mode", (1,)), # switch to manual mode
  }
```

The integers given as arguments to the `switch_mode()` method correspond to the values of the `Mode` enum at the top of the menu file (cf. above).

When you add a new mode, you add its name to the enum and then add a corresponding keystroke-dictionary below the existing ones inside the menu's `__init__()` method. The name of the dictionary must match the one in the enum, except in lower case, so:

```
class Mode(Enum):
    MANUAL = 1
```

corresponds to

```
class Menu():
  def __init__(self):
    # ...
    self.manual = {
      'w': ("robot_controller", "move_man", ('w',)),
      'a': ("robot_controller", "move_man", ('a',)),
      # ...
    }
```

The menu has a checker-function built in that will throw an error if any of the entries you add to a mode-specific dictionary try to use a reserved key. However, you are responsible for making sure you don't try to use the same key twice inside one dictionary.

Imagine you have a key dictionary like this:

```
self.calculator = {
  "a": ("calculator", "sum", (4,3)),
  "a": ("calculator", "prod", (4,2)),
  "s": ("calculator", "square", (2,)),
  "s": ("publisher", "send_string", "Hello, World!")
}
```

You wouldn't get any error message like you would for re-using a reserved key. But if you pressed `a`, the `calculator` node would receive the command `prod()`, and if you pressed `s`, the `publisher` node would receive the command `send_string()`. So keep an eye on that!

A final note on modes: when a node switches modes, its command queue is flushed automatically, i.e. all commands currently in the queue will be discarded. More info on this (and how to override it if that becomes necessary) can be found in the documentation.

Your Task:

For this exercise, we want to implement two new modes: `PRINT` and `MATH`. In `PRINT` mode, the nodes should function just as you have written them, with the `controller` sending a message to be printed to the `actuator`. If the framework is in `MATH` mode, though, the `controller` should instead ask the `actuator` to solve a math problem and print the solution.

- Add modes `MATH` and `PRINT` to the `Mode` enum in `menu.py` and add the corresponding key dictionaries.
- Add two keystroke entries to the dictionary of reserved keys, which send the command to switch to your new modes to the main node.
- Write a function for your `actuator` node that calculates the product of a list of numbers. The list is given as a parameter and the result is printed to the console.

At the moment, you have an entry in the `self.idle` dictionary that calls your `controller` node's `send_string()` method.

- Move this entry from `self.idle` to `self.print`, since you want it to be activated in `PRINT` mode.

You can now take two different approaches. Either, you write write a new function for the `controller` node (e.g. `send_numbers()`) that sends a list of numbers to the `actuator` node along with the command to run the `print_product` method.

- If you choose this option, write a `send_numbers()` method for the controller that send a list of numbers
- Add a entry to the `self.math` dictionary that calls the `controller` node's `send_numbers()` method. **Important**: see the note below on sending a list as a function argument!

Alternatively, instead of writing a separate `send_numbers()` method, you can extend the existing `send_string()` method (even if the name then doesn't make a ton of sense anymore ;9). In this case, you would want the `controller` to send the `print_message` command and a string to the `actuator` when the framework is in `PRINT` mode, and a the `print_product` command and a list of numbers when the framework is in `MATH` mode. (You can check a node's mode like this: `if self.mode == Mode.MATH`.)

- If you choose this option, expand the `send_string()` method you wrote for the controller so it calls `print_product` when the node is in `MATH` mode.
- Add an entry to the `self.math` dictionary that calls the `controller` node's `send_string` function (i.e. you are calling the same function as in `PRINT` mode, but the outcome will be different).

(To use the framework effectively, you will need to be able to implement both versions, so it can't hurt to experiment with them a bit.)

Important Note: If you want to send a tuple or list as the only argument for a key press command or a call to `send_to_queues()`, you need to make sure it's formatted correctly!

Say you have the following method inside a `calculator` node:

```
# method inside "calculator" node:
def sum_tuple(self, numbers:tuple):
  return map(sum, list(numbers))
```

If you now call

```
self.send_to_queues(
  recipients: "calculator",
  msg = ("sum_tuple", [1,2,3])
)
```

from another node that will result in an error. Inside the `Node` class, the `parse_command()` method tries interprets this method as a function name `sum_tuple` and then a list of arguments for that function. This means that sending the command with `[1,2,3]` (or `(1,2,3)`) as the only argument essentially tries to call:

```
sum_tuple(1,2,3)
```

when what you need is

```
sum_tuple([1,2,3])
```

To account for this, make sure that when you are sending a list or tuple as the only argument for a method, you always wrapt it in a second tuple!

```
# Correct:
self.send_to_queues(
  recipients: "calculator",
  msg = ("sum_tuple", ([1,2,3],))
)
```

Also not that there has to be a comma after the list, since Python omits simple brackets around a single value.

```
(4)   == 4          # interpreted as integer
(4,)  == tuple(4)   # interpreted as tuple
```